

Vector Rally - Report

Marta Musso

September 2024

Contents

1	Introduction	3
1.1	Game Rules	3
2	Identified Responsibilities	4
2.1	Model	4
2.2	View	6
2.3	Controller	7
3	SOLID Principles and Patterns Used	8
3.1	SOLID Principles	8
3.1.1	Single Responsibility Principle	8
3.1.2	Open/Closed Principle	8
3.1.3	Liskov Substitution Principle	8
3.1.4	Interface Segregation Principle	9
3.1.5	Dependency Inversion Principle	9
3.2	Patterns Used	9
3.2.1	Factory Pattern	9
3.2.2	Strategy Pattern	9
3.2.3	Singleton Pattern	9
4	Tools Used	10
4.1	Gradle	10
4.2	Javadoc	11
4.3	JUnit Test	12
4.4	JavaFX	13
4.5	Scene Builder	13
4.6	GitHub	13
4.7	Concepts Covered in Class	14
5	Instructions to Run the Project and Useful Links	15
5.1	Run and Build	15

5.2	Useful Links	15
6	Implementation of the Project Specification	16
6.1	Terminal Simulation	16
6.2	Graphical Interface Simulation	17
6.3	Instructions for Simulations	17
7	Conclusion and Future Developments	18
7.1	Future Developments	18

Chapter 1

Introduction

Vector Rally is a turn-based car racing simulation game, designed to be played both through a command-line interface (CLI) and with a simple graphical user interface (GUI) using JavaFX. The game allows players to control their cars on a track, using algorithms to determine valid moves and managing interactions between players and the track.

The game takes place on tracks represented by text files (`.txt`), where each symbol corresponds to a specific track component. For example, symbols can represent the start line and the finish line.

1.1 Game Rules

The game follows the following basic rules:

- **Objective:** The first player to reach the finish line wins.
- **Elimination:** A player without valid moves is eliminated from the match.
- **Moves:** Each turn, players can move their cars based on valid moves determined by the algorithm. Valid moves are determined by the car's current position, its current acceleration, and the track configuration.

Chapter 2

Identified Responsibilities

The Vector Rally project was developed following the Model-View-Controller (MVC) pattern. This approach facilitates the separation of responsibilities, improving code maintainability and extensibility. The main responsibilities and their corresponding classes and interfaces are detailed below.

2.1 Model

The model contains the game logic and represents the data the application works with. It includes the classes that define the players, cars, movements, tracks, algorithms, strategies, and game rules.

- **Defining Players:** Manage the information and actions of players, including their attributes and behaviors.
 - `Player` - Interface for players.
 - `DefaultPlayer` - Abstract class for a default player.
 - `HumanPlayer` - Represents a human player.
 - `BotPlayer` - Represents a bot player.
- **Managing Cars:** Represent players' cars, including colors, states, and movements.
 - `Car` - Interface for cars.
 - `RaceCar` - Implements a race car.
 - `CarColour` - Enumerates car colors.
- **Managing Movements:** Define and validate car movements and their positions on the track.

- **Coordinates** - Interface for the coordinates of a position.
- **Vector** - Interface for a movement vector.
- **Position** - Represents a position on the track.
- **Acceleration** - Represents a car's acceleration.
- **Move** - Record that represents a move.
- **Direction** - Enumerates movement directions.
- **Representing the Track:** Define the various components of the track, such as the start line, finish line, roads, and obstacles. Manage the construction of a track.
 - **Track** - Interface for the track.
 - **RaceTrack** - Represents the race track.
 - **TrackComponent** - Enumerates the track components.
 - **RaceTrackBuilder** - Builds the track from a `.txt` file.
- **Implementing Game Rules:** Establish the game rules, including the validation of moves and interaction with track components.
 - **MoveValidator** - Interface for move validation.
 - **BasicMoveValidator** - Validates basic movements.
 - **BasicMovesGenerator** - Generates basic movements.
- **Implementing Movement Algorithms:** Define and implement algorithms to generate moves based on positions and tracks.
 - **AStar** - Implements the A* algorithm for pathfinding.
 - **Node** - Represents a node in the A* algorithm.
 - **NeighborsGenerator** - Interface for generating neighbors of a position.
 - **FourNeighborsGenerator** - Implements a four-direction neighbor generator.
 - **EightNeighborsGenerator** - Implements an eight-direction neighbor generator.
- **Defining Bot Strategies:** Establish decision-making strategies for bots.
 - **DecisionStrategy** - Interface for bot decision strategies.

- `EasyBotStrategy` - Implements an easy strategy for bots.
- `MediumBotStrategy` - Implements a medium strategy for bots.
- `HardBotStrategy` - Implements a hard strategy for bots.
- `BotStrategy` - Enumerates bot strategies.

2.2 View

The view is responsible for presenting data to the user. It includes the classes that handle input and output.

- **Input/Output Management:** Gather input from the user and display messages and game data.
 - `IOController` - Interface for defining the main I/O actions.
 - `TerminalIOController` - Implementation of I/O for the CLI.
 - `GameMessageProvider` - Implementation for providing game messages.
- **Defining Specific Interactions for Each Game Phase:** Manage the predefined actions of each game phase, from the initial setup to the end of the game, defining the interactions between user and software and how to present them.
 - `SetupGameView` - Interface for defining the main actions regarding the choice and presentation of initial game settings by the user.
 - `CLISetupGameView` - Implementation of the previous interface for the CLI.
 - `MatchGameView` - Interface for defining the behavior and interaction between the user and the game during the match itself, as well as how to present the progress of the match.
 - `CLIMatchGameView` - Implementation of the previous interface for the CLI.
 - `FinishGameView` - Interface for defining the interactions to manage at the end of a match.
 - `CLIFinishGameView` - Implementation of the previous interface for the CLI.

2.3 Controller

The controller manages the flow of the game and the user's interactions with the system. It includes the classes that orchestrate setup operations and match management.

- **Game Setup:** Manage the creation of the game setup, including the list of players and the created track.
 - **GameSetup** - Interface for game setup.
 - **BasicGameSetup** - Generic implementation of game setup (independent of the type of interface used).
 - **SetupResult** - Record resulting from game setup.
- **Match Management and Arbitration:** Control and manage the start of the game, the flow of the match, including conditions and consequences for eliminations and victories, and external management of turns.
 - **MatchController** - Interface for match control.
 - **BasicMatchController** - Generic implementation of match control.
 - **GameEngine** - Interface for game engine control.
 - **BasicGameEngine** - Generic implementation of the previous interface.
 - **TurnHandler** - Interface for turn queue control.
 - **BasicTurnHandler** - Generic implementation of turn control.
- **Building and Controlling Necessary Components:** Control and manage the correct construction and validation of the track, bot strategies, and the list of paths containing files that represent the tracks.
 - **BotStrategyFactory** - Factory for assigning strategies to bots.
 - **TrackBuilder** - Interface for defining track construction.
 - **RaceTrackBuilder** - Implementation of the previous interface using specific components defined in the project.
 - **TrackPathBuilder** - Defines methods for finding the paths of files available to be used as tracks.

Chapter 3

SOLID Principles and Patterns Used

This chapter analyzes the SOLID principles followed during the development of the Vector Rally project, along with the design patterns used.

3.1 SOLID Principles

The SOLID principles are guidelines that help keep the code flexible and maintainable.

3.1.1 Single Responsibility Principle

Each class should have only one responsibility.

- `BasicMoveValidator` has the sole responsibility of validating moves.

3.1.2 Open/Closed Principle

Classes should be open for extension but closed for modification.

- `RaceTrack` can be extended to add new types of track components without altering the basic structure.

3.1.3 Liskov Substitution Principle

Derived classes must be substitutable for their base classes without altering the program's correctness.

- The `Player` interface and its implementations can be used interchangeably in the `handleTurn` method of `BasicMatchController` without altering the game's behavior.

3.1.4 Interface Segregation Principle

Interfaces should be client-specific, avoiding the creation of large and generic interfaces.

- `GameEngine`, `MatchController`, and `TurnHandler` are separate interfaces for managing a match.

3.1.5 Dependency Inversion Principle

Dependencies should be inverted towards abstractions.

- `BasicMatchController` depends on `MatchController` for game configuration, allowing implementation substitutions without modifying the game engine.

3.2 Patterns Used

In the Vector Rally project, several design patterns were used to improve the code structure, making it more flexible and extensible.

3.2.1 Factory Pattern

The Factory Pattern is used to create objects without exposing the creation logic to the client. This pattern is employed in the `BotStrategyFactory` class to create bot strategies.

3.2.2 Strategy Pattern

The Strategy Pattern is used to define a family of algorithms, encapsulate them, and make them interchangeable. This pattern is used to determine the decision strategies of bots.

3.2.3 Singleton Pattern

The Singleton Pattern is used to ensure that there is only one instance of `SceneManager` during the application's execution.

Chapter 4

Tools Used

This chapter describes the tools used during the development of the Vector Rally project. Each section provides an overview of the tool and code examples to illustrate its use.

4.1 Gradle

Gradle was used to manage dependencies and automate the build process of the project. Thanks to Gradle, it was possible to easily configure the necessary libraries and manage compilation, execution, and test tasks.

Example of `build.gradle` file:

```
plugins {  
    id 'application'  
    id 'org.openjfx.javafxplugin' version '0.1.0'  
    id("buildlogic.java-application-conventions")  
}  
  
repositories {  
    mavenCentral()  
}  
  
run {  
    standardInput = System.in  
}  
  
dependencies {  
    implementation 'org.apache.commons:commons-text:1.10.0'
```

```

        implementation 'org.openjfx:javafx-controls:18.0.1'
        implementation 'org.openjfx:javafx-fxml:18.0.1'
        implementation project(':api')
    }

    java {
        toolchain {
            languageVersion = JavaLanguageVersion.of(21)
        }
    }

    javafx {
        version = "22.0.1"
        modules = [ 'javafx.controls', 'javafx.fxml' ]
    }

    application {
        mainClass = 'it.unicam.cs.mpmgc.vectorrally.app.GraphicalApp'
    }

```

4.2 Javadoc

Javadoc was used to document the code. By generating HTML documentation directly from the source code, it is possible to keep the documentation always up to date with the code's evolution.

Example of a Javadoc comment:

```

/**
 * Retrieves the component at the specified position on the track.
 *
 * @param x the x coordinate.
 * @param y the y coordinate.
 * @return the {@link TrackComponent} at the specified position.
 */
TrackComponent getComponentAt(int x, int y);

```

4.3 JUnit Test

JUnit was used to test the code. Thanks to JUnit, it is possible to write automated tests to verify the correctness of the implemented functionalities and ensure that code changes do not introduce bugs.

Example of a JUnit test:

```
public class BotPlayerTest {

    private BotPlayer botPlayer;
    private Car car;
    private BotStrategy strategy;

    @BeforeEach
    void setUp() {
        car = new RaceCar(CarColour.BLUE);
        strategy = BotStrategy.EASY;
        botPlayer = new BotPlayer(car, strategy);
    }

    @Test
    void constructorShouldThrowExceptionWhenCarIsNull() {
        assertThrows(NullPointerException.class,
            () -> new BotPlayer(null, strategy));
    }

    @Test
    void constructorShouldThrowExceptionWhenStrategyIsNull() {
        assertThrows(NullPointerException.class,
            () -> new BotPlayer(car, null));
    }

    @Test
    void getStrategyShouldReturnStrategy() {
        assertEquals(strategy, botPlayer.getStrategy());
    }
}
```

4.4 JavaFX

JavaFX was used to create the game's graphical interface. It provides a rich library of UI components and tools to build modern and interactive user interfaces.

Example of using JavaFX:

```
@FXML
public void initialize() {
    nextTurnButton.setOnAction(event -> goToNextTurn());
    turnLabel.setText(messageProvider.getPreparationMessage());
}
```

4.5 Scene Builder

Scene Builder was used to design user interfaces in JavaFX. It allows building scenes visually and generating the corresponding FXML file, facilitating the separation between logic and presentation.

Example of FXML code generated with Scene Builder:

```
<rowConstraints>
    <RowConstraints minHeight="10.0" prefHeight="30.0" />
    <RowConstraints minHeight="10.0" prefHeight="30.0" />
    <RowConstraints minHeight="10.0" prefHeight="30.0" />
</rowConstraints>
```

4.6 GitHub

GitHub was used for version control and project development across its various implementation levels. Thanks to GitHub, it is possible to track changes and manage development branches.

In particular, 3 different branches were used to manage the project's development in its different stages:

- main
- development_base
- development_GUI

4.7 Concepts Covered in Class

During the development of the project, several constructs and concepts covered in class were used, such as the use of interfaces and records.

Example of a record:

```
public record SetupResult(  
    NeighborsGenerator generator ,  
    RaceTrack raceTrack ,  
    List<Player> players) {  
}
```

Example of Generics:

```
public class BasicMovesGenerator<T extends NeighborsGenerator> {  
    private final T neighborsGenerator ;  
    private final MoveValidator moveValidator ;  
}
```

Chapter 5

Instructions to Run the Project and Useful Links

5.1 Run and Build

To build and run the project, use the following commands:

```
gradle build
gradle run
```

The graphical interface will be displayed. For terminal simulation, run `TerminalApp` from the `app` module or modify the `build.gradle` file of the `app` module as follows:

```
application {
    mainClass = 'it.unicam.cs.mpmgc.vectorrally.app.GraphicalApp'
}

application {
    mainClass = 'it.unicam.cs.mpmgc.vectorrally.app.TerminalApp'
}
```

5.2 Useful Links

A reference to the [Github Repository](#) containing the Vector Rally project code is provided.

Chapter 6

Implementation of the Project Specification

The Vector Rally project allows you to run the game simulation both from the terminal and through a graphical interface, as described in Chapter 5.

6.1 Terminal Simulation

The terminal simulation has been carefully designed in many aspects, also providing interaction for a human user. The terminal simulation has been developed to be quite complete and sufficiently detailed, occupying most of the project development time.

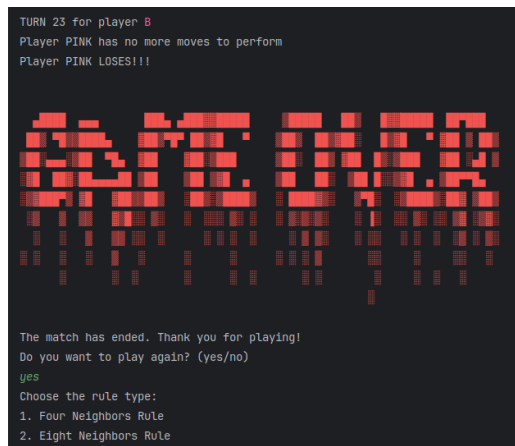


Figure 6.1: Defeat during a terminal match

6.2 Graphical Interface Simulation

The graphical interface simulation also offers a sufficient level of interaction, allowing the user to select the move directly on the track.



Figure 6.2: Initial match setup on graphical interface

6.3 Instructions for Simulations

For both simulations, just follow the instructions that are shown. The terminal simulation will provide the necessary prompts to interact with the game. For the graphical interface simulation, you need to start the game, configure the match settings, and start the game simulation. To interactively select the desired move, simply click on one of the gray rectangles that represent the available destinations.

Chapter 7

Conclusion and Future Developments

The code has been designed to be extensible and modular, facilitating the addition of new features and future improvements.

7.1 Future Developments

The Vector Rally project was developed with the goal of creating modular and extensible code. This approach allows adding new features without significantly modifying the existing code, ensuring greater flexibility and maintainability. Among the planned future developments are:

- **Improvement of bot AI algorithms:** Testing additional algorithms that allow the search for minimum paths to make the competition more interesting.
- **Advanced Graphical Interface:** Making the graphical interface more advanced and attractive, including features such as interactive loading of new tracks and more intuitive visual feedback.